

Computational Complexity of Sorting

CS1200: Introduction to Algorithms and their Limitations

Lecture 3

Fall 2026

Harvard SEAS

Put away your devices

Where we are

- **Lecture 1:** three sorting algorithms — ExhaustiveSearchSort, InsertionSort, MergeSort — and proofs that they are *correct*.
- **Lecture 2:** what a *computational problem* is (Def. 3.4), what it means for an algorithm to *solve* a problem, the definitions of the running time $T(n)$ and asymptotic growth, and SingletonBucketSort.

Today:

1. Compute $T(n)$ for all four sorting algorithms and compare algorithms using *asymptotic notation*.
2. Explore the difference between bounding *algorithms* and bounding *problems*.

The *computational complexity* of a problem Π is the smallest growth rate achievable by any algorithm **in a given model** that solves Π .

“Any algorithm” is meaningless until we say what an algorithm is *allowed to do*.

Asymptotic notation

Big O and Big Ω

Definition 3.7

Let $h, g : \mathbb{N} \rightarrow \mathbb{R}_{\geq 0}$. We write $h = O(g)$ if there is a constant $c > 0$ such that for all sufficiently large n , $h(n) \leq c \cdot g(n)$.

Pay close attention to the quantifiers:

$$\exists c > 0 \ \exists n_0 \ \forall n \geq n_0 : h(n) \leq c g(n).$$

Example. $h(n) = 3n^2 + 2n + 5$, $g(n) = n^2$: $3n^2 + 2n + 5 < 3.1 n^2$ for all sufficiently large n , so take $c = 3.1$ and $h(n) = O(n^2)$.

Definition 3.8 (big Omega)

$h = \Omega(g)$ if there is a constant $c > 0$ such that for all sufficiently large n , $h(n) \geq c \cdot g(n)$.
(Equivalently, $g = O(h)$.)

Computing $T(n)$

$T(n)$ and basic operations

Informal Definition 3.5 (worst-case running time)

For an algorithm A with input set I and size function $\text{size} : I \rightarrow \mathbb{N}$, the running time is $T : \mathbb{R}_{\geq 0} \rightarrow \mathbb{N}$,

$$T(n) = \max_{x \in I : \text{size}(x) \leq n} (\# \text{ basic operations } A \text{ performs on } x).$$

- $\leq n$, not $= n$: the max is over *all* smaller sizes, which forces T to be **nondecreasing**.
- **Domain** $\mathbb{R}_{\geq 0}$: lets us write $T(n/2)$ even when $n/2$ is not an integer.

What operations will we count?

comparisons between keys, and **copies** of key–value pairs. Every other step these algorithms take is at most a constant times these — so Θ does not change.

ExhaustiveSearchSort

ExhaustiveSearchSort(A)

Input: array $A = ((K_0, V_0), \dots, (K_{n-1}, V_{n-1}))$, each $K_i \in \mathbb{R}$

Output: a valid sorting of A

```
0  foreach permutation  $\pi: [n] \rightarrow [n]$  do
1    if  $K_{\pi(0)} \leq K_{\pi(1)} \leq \dots \leq K_{\pi(n-1)}$  then
2      return  $((K_{\pi(0)}, V_{\pi(0)}), \dots, (K_{\pi(n-1)}, V_{\pi(n-1)}))$ 
3  return  $\perp$ 
```

- Line 0 loops $n!$ times — there are $n!$ permutations of $[n]$.
- Line 1 is **not** a single basic operation. It is $n - 1$ comparisons, i.e. a loop: $O(n)$ per iteration.

•

$$T_{\text{exhaustsort}}(n) = n! \cdot O(n) = O(n! \cdot n).$$

- **Lower bound:** for some input the sorted order is the *last* permutation tried, so all $n!$ iterations run — $T_{\text{exhaustsort}}(n) = \Omega(n! \cdot n)$, hence $\Theta(n! \cdot n)$.

In groups: InsertionSort

InsertionSort(A)

```
0  /* in-place: modifies A until it is sorted */
1  foreach  $i = 0, \dots, n-1$  do
2    /* insert  $A[i]$  among  $A[0], \dots, A[i-1]$  */
3    Find the first index  $j$  with  $A[i][0] \leq A[j][0]$ 
4    Insert  $A[i]$  at position  $j$ , shift  $A[j \dots i-1]$  right
5  return A
```

In iteration i , say line 3 stops at index j .

(Always $j \leq i$: $A[i]$ qualifies against itself.)

1. Line 3 makes $j + 1$ comparisons.
2. Line 4 makes $i - j + 1$ copies (shift $i - j$, then place $A[i]$).

The j 's cancel: every iteration costs exactly $i + 2$ steps, whatever the input.

InsertionSort: adding it up

Iteration i costs exactly $i + 2$, so

$$T_{\text{insertsort}}(n) = \sum_{i=0}^{n-1} (i + 2) = \sum_{i=0}^{n-1} (i + 1) + n.$$

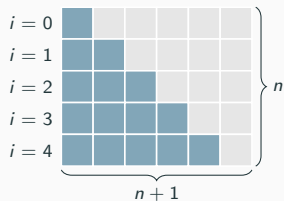
That sum is a triangular number:

$$\sum_{i=0}^{n-1} (i + 1) = 1 + 2 + \cdots + n = \frac{n(n+1)}{2}.$$

So

$$T_{\text{insertsort}}(n) = \frac{n(n+1)}{2} + n = \Theta(n^2).$$

Lower bound: as written, every input costs $\Omega(n^2)$, even sorted. A right-to-left scan fixes sorted input, but on reversed input each $A[i]$ still **compares** with and **passes** all i earlier entries: $\Omega(n^2)$ regardless.



Two copies of $1 + 2 + \cdots + n$ fill an $n \times (n + 1)$ rectangle.

MergeSort: setting up the recurrence

MergeSort(A)

```
0  if  $n \leq 1$  then return  $A$ 
1  else
2     $i = \lceil n/2 \rceil$ 
3     $B = \text{MergeSort}(A[0 \dots i - 1])$ 
4     $B' = \text{MergeSort}(A[i \dots n - 1])$ 
5    return Merge( $B, B'$ ) — compares and copies
```

Merge on two sorted arrays of total length m makes $\leq m$ comparisons and exactly m copies, so $T_{\text{merge}}(m) = \Theta(m)$.

An input of size n splits into halves of size $\lceil n/2 \rceil$ and $\lfloor n/2 \rfloor$, sorts each, then merges:

$$\begin{aligned} T_{\text{mergesort}}(n) &\leq T_{\text{mergesort}}(\lceil n/2 \rceil) + T_{\text{mergesort}}(\lfloor n/2 \rfloor) + T_{\text{merge}}(n) + \Theta(1) \\ &= T_{\text{mergesort}}(\lceil n/2 \rceil) + T_{\text{mergesort}}(\lfloor n/2 \rfloor) + \Theta(n). \end{aligned}$$

Unrolling, when n is a power of 2

$$\begin{aligned}T_{\text{mergesort}}(n) &\leq 2 T_{\text{mergesort}}(n/2) + cn \\&\leq 2(2 T_{\text{mergesort}}(n/4) + c(n/2)) + cn = 4 T_{\text{mergesort}}(n/4) + 2cn \\&\leq 4(2 T_{\text{mergesort}}(n/8) + c(n/4)) + 2cn = 8 T_{\text{mergesort}}(n/8) + 3cn \\&\vdots \\&\leq 2^\ell T_{\text{mergesort}}(n/2^\ell) + \ell cn \quad \text{for any } \ell \leq \log n.\end{aligned}$$

How far do we unroll? Until the pieces hit the base case, size 1: $n/2^\ell = 1 \iff \ell = \log n$.

$$T_{\text{mergesort}}(n) \leq n \cdot \underbrace{T_{\text{mergesort}}(1)}_{\Theta(1): \text{ line 0 just returns}} + c n \log n = O(n) + O(n \log n) = O(n \log n).$$

n not a power of 2? Let n' be the least power of 2 with $n' \geq n \geq n'/2$. Then

$$T_{\text{mergesort}}(n) \leq T_{\text{mergesort}}(n') = O(n \log n).$$

Lower bound: Merge must **copy** all of its output, so each of the $\lceil \log n \rceil$ levels costs $\geq n$: $\Omega(n \log n)$.

Theorem 3.10

Theorem 3.10 (informal)

The worst-case runtimes of ExhaustiveSearchSort, InsertionSort, and MergeSort are $\Theta(n! \cdot n)$, $\Theta(n^2)$, and $\Theta(n \log n)$ respectively.

category	form	exemplar
exponential (slow)	$T(n) = 2^{n^{O(1)}}$	ExhaustiveSearchSort
polynomial (reasonable)	$T(n) = n^{O(1)} = \text{poly}(n)$	InsertionSort
nearly linear (fast)	$T(n) = O(n \log n)$	MergeSort

The first limitation

Sorting: both bounds

Upper bound: **some** algorithm achieves $O(n \log n)$

Build one algorithm.

- The algorithm *is* the proof. It is an existence claim.
- No model needed.

Lower bound: **all** algorithms need $\Omega(n \log n)$

Rule out every algorithm.

- Define the class of possible algorithms **precisely**, then prove every member needs $\Omega(n \log n)$ steps.

We can't count steps in a proof of non-existence.

A model lets us reason about the steps needed by all possible algorithms.

The catch

A narrower model makes it *easier* to write a proof, but it proves a weaker claim.

Definition: comparison-based sorting

A sorting algorithm is *comparison-based* if the only way in which it uses the keys is by comparing them. Comparison-based sorting can be modeled by using a special data type `key`.

Keys as a data type

Keys have their own data type `key`. The only permitted operations on variables `var`, `var'` of that type are:

- `var = var'` (copy)
- `var ≤ var'` (compare, returns a boolean)

Comparison-based programs are not allowed to convert between type `key` and other data types (like `int`) to perform other operations on them (like arithmetic operations).

The theorem

Theorem 3.11

If A is a comparison-based algorithm that correctly solves SORTING on arrays of length n in time $T(n)$, then

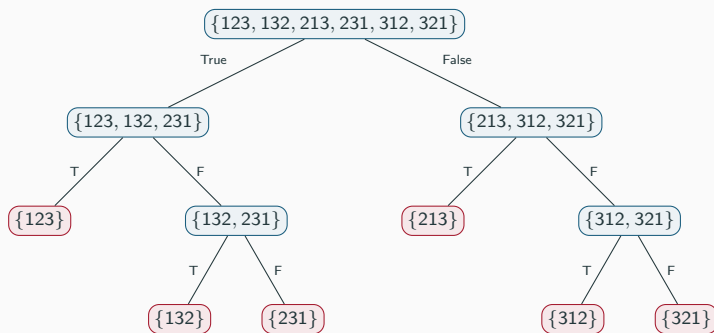
$$T(n) = \Omega(n \log n).$$

Moreover this holds even if the keys are restricted to lie in $[n]$ and all values are empty.

What this says

MergeSort is asymptotically *optimal* among comparison-based sorting algorithms. No matter how clever computer scientists are in the future, there will be no asymptotically faster comparison-based sort.

The idea: the algorithm's knowledge is a tree



Each node = a point in the execution. The set = the orderings still consistent with every comparison made so far. Each edge = one comparison outcome.

The proof, by counting

A receives input I , an array of n key–value pairs in some order, and must return a rearrangement $\pi(I)$ that is sorted. Consider inputs whose n keys are distinct: there are $n!$ orderings of them, and each needs a different π , so A must distinguish $n!$ cases.

- The permutation π that A applies depends **only** on the outcomes of its key comparisons. A has no other information about the input.
- A makes at most $T(n)$ comparisons, each with 2 outcomes. So it has at most $2^{T(n)}$ behaviors available, and needs $n!$ of them.

We need:

$$2^{T(n)} \geq n!$$

Too few behaviors $\Rightarrow$ two orderings get treated identically $\Rightarrow$ one comes out unsorted.

So how many comparisons do we need?

We need $2^{T(n)} \geq n!$. Try some values of T :

T	behaviors available	enough for $n!$?
n	2^n	no — already $2^4 = 16 < 24 = 4!$
$n \log n$	$2^{n \log n} = n^n$	yes — and MergeSort achieves it

So the answer sits somewhere between n and $n \log n$. And n^n is a great deal *bigger* than $n!$ — so is there room to do better than $n \log n$?

Stop guessing. Solve for T .

$$2^{T(n)} \geq n! \quad \implies \quad T(n) \geq \log_2(n!)$$

One step left: how big is $\log_2(n!)$?

In groups: how big is $\log_2(n!)$?

We have $T(n) \geq \log_2(n!)$.

Show that $\log_2(n!) = \Omega(n \log n)$.

Hints

1. A logarithm turns a product into a sum: $\log_2(n!) = \log_2(1 \cdot 2 \cdots n) = \sum_{k=1}^n \log_2 k$.
2. Every term is ≥ 0 , so *discarding* terms only makes the sum smaller, so a lower bound on a subset of terms is still a lower bound on the whole thing.
3. Discard all terms except those with $n/2 \leq k \leq n$. **How many terms are left? How small can each one be?** Multiply those two answers.

Answer

Discard every term with $k < n/2$ — all terms are ≥ 0 :

$$\log_2(n!) = \sum_{k=1}^n \log_2 k \geq \sum_{k \geq n/2} \log_2 k \geq \underbrace{(n/2)}_{\text{that many terms}} \cdot \underbrace{\log_2(n/2)}_{\text{each at least this}}.$$

Now just simplify, using $\log_2(n/2) = \log_2 n - 1$:

$$\begin{aligned}(n/2) \cdot \log_2(n/2) &= (n/2) (\log_2 n - 1) \\ &= (1/2) n \log_2 n - n/2 \\ &= \Omega(n \log n).\end{aligned}$$

The leading term is $(1/2) n \log_2 n$; what we subtract is only *linear*.
(Explicitly: $(1/2) n \log_2 n - n/2 \geq (1/4) n \log_2 n$ once $n \geq 4$ — so $c = 1/4$, $n_0 = 4$.)

Theorem 3.11

$T(n) \geq \log_2(n!) = \Omega(n \log n)$, so every comparison-based sorting algorithm has $T(n) = \Omega(n \log n)$. ■

Escaping the lower bound

A different problem statement

Computational Problem: Sorting On Finite Universe

Input: a universe size $U \in \mathbb{N}$ and an array $A = ((K_0, V_0), \dots, (K_{n-1}, V_{n-1}))$ with every key $K_i \in [U]$.

Output: a valid sorting of A .

Every sorting algorithm we have applies to this directly — keys in $[U]$ are still just keys. So MergeSort already solves it in $O(n \log n)$.

Theorem (Section 3.6)

There is an algorithm for SORTING ON FINITE UNIVERSE with worst-case running time $O(n + U)$. One such algorithm is SingletonBucketSort. With $U = n$: $O(n + n) = O(n)$.

We proved an $\Omega(n \log n)$ lower bound for sorting — *even for keys in $[n]$* . **Is this a contradiction?**

Asymptotics with two parameters

We want to say $T(n, U) = O(n + U)$. What does that mean?

There are constants $c > 0$ and n_0 such that

$$T(n, U) \leq c(n + U) \quad \text{for every } n, U \text{ with } n + U \geq n_0.$$

Equivalently: the inequality is allowed to fail for only **finitely many** pairs (n, U) — the small ones.

(Which is also an equivalent way to state the *one*-parameter definition: “for all sufficiently large n ” means “for all but finitely many n .”)

SingletonBucketSort

SingletonBucketSort(U, A)

Input: $U \in \mathbb{N}$, array A with all $K_i \in [U]$

Output: a valid sorting of A

- 0 Initialize an array C of length U ; each entry is an empty linked list
- 1 foreach $i = 0, \dots, n - 1$ do
- 2 Insert (K_i, V_i) into linked list $C[K_i]$ as the new *head*
- 3 Form $A' =$ elements of $C[0]$, then $C[1]$, then $C[2]$, ...
- 4 return A'

Refresher: the same trace as last class

$$U = 5, \quad A = ((2, a), (3, b), (0, c), (4, d), (3, e), (3, f), (0, g)).$$

$C[0]$	$(0, g) \rightarrow (0, c)$
$C[1]$	—
$C[2]$	$(2, a)$
$C[3]$	$(3, f) \rightarrow (3, e) \rightarrow (3, b)$
$C[4]$	$(4, d)$

$$A' = (0, g), (0, c), (2, a), (3, f), (3, e), (3, b), (4, d)$$

Claim (after the loop)

After the loop, for each $K \in [U]$, the linked list $C[K]$ contains exactly the key-value pairs of A whose key equals K .

“Valid sorting of A ” is a **two**-part specification — so the proof has two parts.

- **A' is a permutation of A .** Each pair was inserted into exactly one list (namely $C[K_i]$), and nothing else was ever inserted. So concatenating the lists reproduces every pair exactly once.
- **Its keys are ascending.** Every pair in $C[K]$ has key exactly K , and line 3 walks $C[0], C[1], \dots, C[U - 1]$ in increasing order of K .

In groups: count the steps

SingletonBucketSort(U, A)

```
0  Initialize an array  $C$  of length  $U$ ; each entry an empty list
1  foreach  $i = 0, \dots, n-1$  do
2    Insert  $(K_i, V_i)$  into linked list  $C[K_i]$  as the new head
3  Form  $A' =$  elements of  $C[0]$ , then  $C[1]$ , then  $C[2]$ , ...
```

How many steps does each part take?

1. **Line 0:** $O(U)$ — create U empty lists.
2. **Lines 1–2:** n iterations, $O(1)$ each — $O(n)$.
3. **Line 3:** $O(|C[j]| + 1)$ per bucket, so $\sum_j O(|C[j]| + 1) = O(U + n)$, since $\sum_j |C[j]| = n$. The $+1$: an *empty* bucket still costs a step.

$$T_{\text{sbs}}(n, U) = O(U) + O(n) + O(n + U) = O(n + U).$$

In groups: So where is the contradiction?

Theorem 3.11

Every comparison-based sort takes $\Omega(n \log n)$, even for keys in $[n]$.

SingletonBucketSort

Sorts keys in $[U]$ in time $O(n + U)$. With $U = n$, that is $O(n)$.

No contradiction

SingletonBucketSort is *not comparison-based*. It never compares keys. It uses K_i as an **index into an array**, which is forbidden in the comparison-based model.

When is SingletonBucketSort better?

- **Reading.** Hesterberg–Vadhan §3.2–3.6.
(§3.4's proof is optional reading in the book — but we did it, so you have seen it.)
- **Next time.** Reductions, and static data structures. Chapters 4 and 5.
- **Do now:** Problem Set 0 in-class follow-up.